

Automated Live Migration of Containerised Applications in Multi Cloud with Algorithm for Selection of Cloud Provider

A. Sakthe Balan, Abhiraj, Adithya Ganesh, Advay Agrawal, Dr. Prafullata K. Auradkar

Centre for Cloud Computing and Big Data

Department of Computer Science

PES University

Bengaluru 560085, India

Emails: sakthebalan2003@gmail.com, abhiraj.official15@gmail.com, adithyag020@gmail.com, advay1407@gmail.com

Abstract—Efficiently migrating containerized applications across multiple cloud providers presents several challenges, with selecting an optimal cloud service provider and the lack of live migration capabilities in popular container engines docker being among the most significant. Migrating containers between different cloud service providers allows for a much greater amount of freedom for the users and takes a step towards avoiding vendor lock-in. This paper introduces a novel algorithm that optimizes cloud service provider selection using real-time metrics, allowing users to prioritize factors such as cost and latency for a more flexible and efficient hosting solution. Additionally, our approach enables seamless iterative live migration with minimal downtime while extending support to widely used container engines like Docker using pre-copy checkpoints. By integrating load balancing, VPN tunneling, and location-aware search space optimization, we enhance multi-cloud transitions for containers.

I. Introduction

Hosting production applications on third-party cloud service providers (CSPs), aiming for 100% availability, can prove to be extremely costly in the long run. Observing costs of VM instances on different CSPs there is a little difference in the per hour price, but this price difference can accumulate and grow exponentially over long periods of time. The problem of vendor lock-in also arises in this context, where users become entirely dependent on a single CSP thus resulting in high expenses at the same time making migration difficult, further reinforcing the need for multi-cloud migration solutions. Migrating containerized programs across cloud platforms is a significant difficulty in modern distributed computing. Live migration is the process of migrating an application from one physical or virtual host to another with minimal downtime. This ensures service continuity and availability for the maximum length of time. However, in the context of containerized environments, this process is inherently complex due to the need to preserve the application memory state along with the file system that has been used by the container application to persist data

across runs, in addition to this, interconnected containers further add complexity.

The core challenge addressed in this paper is twofold: firstly, to search for a suitable replacement CSP and instance type to optimize the performance and hosting costs of the application based on the user's priorities. While hosting latency sensitive applications, cost efficiency might not be of first priority and lower latency at a higher cost might be desirable, hence the option for the user to prioritize between the two is required. Factors such as cost efficiency, latency, resource availability, and geographical proximity play crucial roles in the decision-making process, and this paper presents a solution that automates this decision-making process. The proposed system is designed to automatically monitor and retrieve relevant metrics from various CSPs, using this data to dynamically optimize the selection of a target cloud provider.

And secondly, to provide a mechanism for seamless live migration of containers between CSPs based on the user defined priorities and parameters while extending the support for live migration to a larger subset of container engines being widely used in production applications. The live migration process itself involves several key technologies, including container runtime environments (RunC), checkpoint/restore mechanisms (CRIU), and data transfer protocols (rsync). Through the integration of these tools, the system ensures that container state—including memory, file system, and network connections—is preserved throughout the migration. Additionally, we explore migration within Docker environments (an area not extensively studied) by expanding iterative migration support from lower-level technologies like RunC to more popular environments such as Docker and Containerd, thereby enhancing the flexibility and applicability of the solution.

This paper will explore the architectural design and the system requirements that make such a solution feasible. We will begin with a brief background on the Docker architecture and the inter-connectivity of related components, which will be followed by comprehensive literature review, examining current techniques for live migration, and the state of multi-cloud environments. Next section will contain an in-depth discussion of the system's high-level design, including the

tools and technologies employed, the novel aspects of the migration process, and the optimization strategies used to minimize downtime. The implementation section will detail the code changes and algorithms employed, while the results section will present data from simulations and tests that demonstrate the system’s efficiency along with a brief discussion over the difference in costs for different types of instances for different CSPs

II. Background

II.1. Docker Architecture

Docker’s architecture contains several important elements that interact and work together for the management, running, and deployment of containers. The essential components of this architecture include Docker CLI, dockerd, containerd, and runc. All these are used by the current experimental mechanism of Docker checkpoint. The overall flow of communication starts from Docker CLI where the user can give a docker command, CLI issues a request to the docker daemon moby running as a system service as a REST API request, which in turn calls containerd via internal gRPC calls. Figure 1 gives a great overview of the hierarchical architecture followed by Docker and its components. The following is a more in-depth view at the different components and how they are connected.

Docker CLI Docker CLI (docker) is the command-line interface tool for users to interact with Docker. It sends commands like `docker run`, `docker build`, and `docker pull` to the Docker daemon, dockerd. The CLI is, in addition, the main user-facing component: it will enable users to manage containers, images, networks, and volumes. To support interactions with a remote daemon as well, internally, the CLI makes use of REST API calls to communicate and issue requests towards the docker daemon.

Docker Daemon (dockerd) Docker Daemon (dockerd) now also known moby is the core background service responsible for managing Docker containers and images. It runs as a system service and listens for requests from the CLI. dockerd serves as the highest level abstraction for managing docker container. It handles high-level operations like image management, container creation, and networking, while delegating some lower-level operations to containerd. dockerd requires containerd on the same machine. Both communicate with each other via Google Remote Procedure Calls (gRPCs).

Containerd Containerd is an industry-standard container runtime and life cycle manager that Docker relies on to handle container operations at a lower level than Dockerd. Unlike Docker, which provides a high-level interface, Containerd focuses on basic container management functionalities like creating, starting, and stopping containers, making it suitable for integration within other systems instead of production use.

Runc The actual low-level runtime responsible for spawning and running containers according to the Open

Container Initiative (OCI) specifications is Runc. It is used in Containerd to build up namespaces, cgroups and other kernel isolation mechanisms. And when a container is configured it calls Runc which will run that container. Well, it would be the plumbing to make cross-platform containerization without having to recreate the same old wheel at each turn.

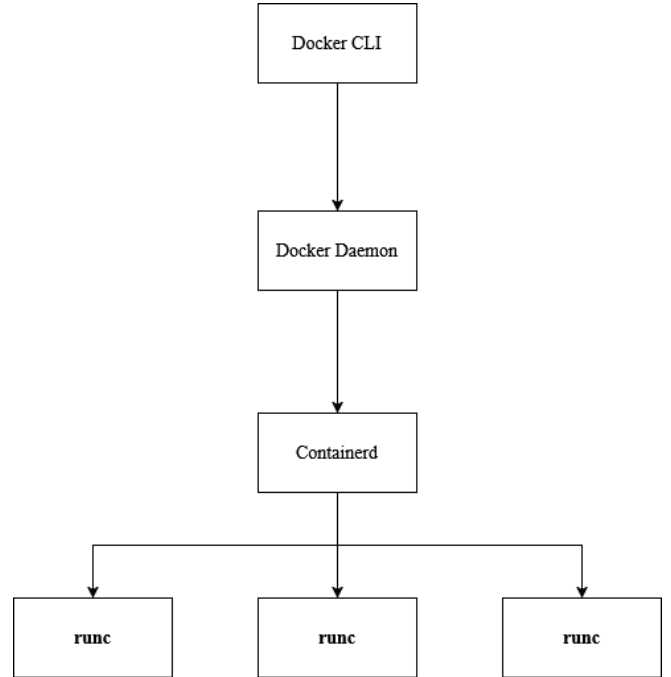


Figure 1. Docker Architecture

III. Related Work

In this section, we discuss in-depth about the previous work that has been done in the same field.

Due to the ever growing popularity of containers in today’s cloud market due to their efficiency and lightweight nature. Containerized application migration has become an essential focus, driven by the need for minimal application downtime, high scalability, and enhanced resource management. This section discusses existing contributions, particularly [7]CloudHopper, which address critical aspects of live container migration for Runc .

The paper [6]Efficient Live Migration of Linux Containers introduces novel techniques to optimize the live migration of Linux containers using Checkpoint-Restore in Userspace (CRIU). The primary challenge addressed is minimizing downtime and total migration time, particularly for memory-intensive applications. This work highlights the inefficiencies of traditional disk-based migration and proposes the use of CRIU’s image-cache and image-proxy mechanisms to bypass persistent storage, significantly reducing I/O overhead. The authors evaluate three major migration algorithms: pre-copy, post-copy, and a hybrid

approach combining the two. Pre-copy migration iteratively transfers memory pages while the container is still running, reducing downtime but potentially increasing total migration time for write-intensive applications. Post-copy migration, in contrast, minimizes downtime by transferring memory on demand after the container is restored at the destination, though it risks performance degradation due to network latency. The hybrid approach balances these trade-offs by pre-copying frequently accessed memory pages and leveraging post-copy for the remainder, achieving both lower downtime and improved reliability.

[7]CloudHopper, on the other hand, extends container migration capabilities utilizing the OCI compliant runtime of runc for multi-cloud environments, enabling the seamless movement of runc containerized applications across Amazon Web Services(AWS), Google Cloud Platform(GCP), and Microsoft Azure. CloudHopper focuses on inter-cloud mobility by employing pre-copy optimizations, connection holding, and traffic redirection mechanisms. Its ability to support multi-container setups and maintain zero client-perceived downtime highlights its suitability for real-world commercial cloud systems. By automating the migration process and incorporating advanced scheduling and parallel migration techniques, CloudHopper addresses the challenges of latency and interdependence in multi-container web applications. Cloudhopper presents its results for MySQL and Wordpress containers on varying loads.

[1]Voyager Complete Container State Migration presents a just-in-time live migration service tailored for containerized environments. By combining CRIU-based memory migration with union-mounted data federation, [1]Voyager minimizes downtime and supports file system agnostic migrations. Its ability to enable zero-copy transfers during migration is particularly relevant to our focus on enhancing resource efficiency and maintaining service continuity during container transitions.

These works collectively address critical aspects of container migration based on different container runtimes and built for different purposes. By leveraging insights from these contributions, our research aims to develop a hybrid migration framework that focuses on several parameters including cost-efficiency, and latency sensitivity among several others. We aim to provide a wider engine support by enabling migration of production ready docker containers, instead of low level runc containers, which has not been attempted yet along with algorithmically backed solution through a user-intuitive UI for the users to search, select and initiate migration of their containerized application across different cloud providers.

IV. Design and Architecture of Solution

IV.1. Overview

Figure 2 gives a great overview for the overall flow of the solution. The entire solution. As illustrated, the entire solution is divided into two parts, the frontend and the backend.

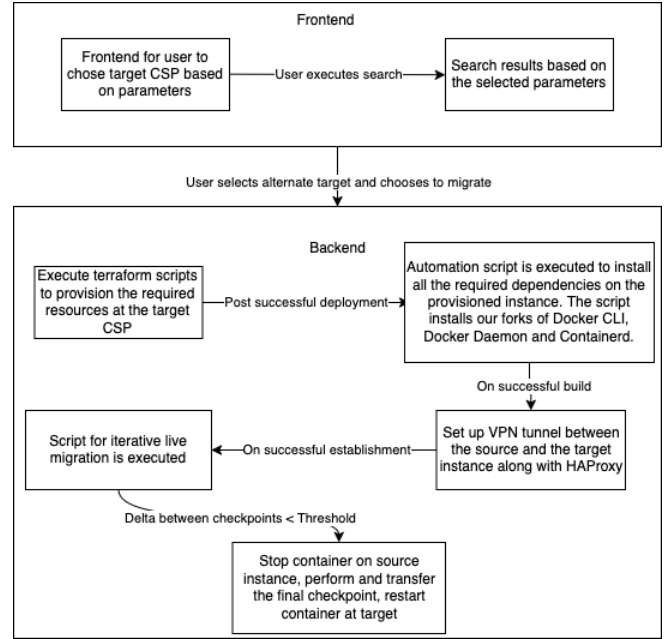


Figure 2. Complete flow of the solution

The user interacts with the frontend in order to search for suitable target Cloud Service Providers (CSPs). The backend is where the majority share of the tasks execute, beginning from the provision of target cluster to the iterative live migration process. The following section goes into detail about the architecture and design of each component and task being executed.

IV.2. Frontend

IV.2.1. Web Application. The user is provided with an intuitive User Interface (UI) in order to streamline the process of migration. First step includes tuning the input parameters to the program, which identifies a more suitable Cloud Service Provider (CSP) as an alternative to the current one, if available. This includes the selection of Upon confirmation of the migratable CSP platform, the algorithm initiates a process that encompasses the entire logic of live migration.

IV.3. Backend

IV.3.1. Provisioning Target Instance. The configuration of the infrastructure is managed with Terraform. Based on user selection, a dynamic terraform file is generated which is used for automated instance provisioning and dependency installation. Using two connected files where one is a file for initializing parameters along with their default values which includes the region of the CSP (default: us-east-1), ID of the instance type (AMI for an EC2 in the case of AWS as an example), type of instance, specifications of the subnet, and configurations of SSH key for secure access while the other file would be the execution blueprint using those variables to actually create the infrastructure.

Once the infrastructure setup is successful, the main terraform script creates a provisioning sequence. It sends a shell script to the newly developed instance using a file provisioner. This places the script in the file system of the newly provisioned instance. The remote-exec provisioner makes the script executable and runs the script. This installs the dependencies and prepares the environment for running the docker application that will be migrated from the source.

IV.3.2. Setting up Target Instance. : As mentioned in the previous section a shell script that orchestrates automated environment setup is transferred and executed on the target instance.

The script first follows a structured installation, which installs system-level dependencies: git, make, gcc, wget, and security development libraries using the apt package manager. The next step of the environment setup is installing Go language, which is pivotal to building the Docker components. We utilize the snap package manager to install Go and establish the required environment variables such as GOPATH. After all dependencies are met the script clones three repositories that have been forked from open source official repositories by us, including a forked Docker CLI, Docker daemon, or moby, and containerd.

This will build the three core components: Containerd is compiled from source, and then a detailed configuration file defines multiple plugin settings, runtime parameters, and security policies. The Docker daemon (dockerd) is built from the moby source code, installed into the system, and then configured to run as a systemd service. Lastly, the Docker CLI is compiled and installed to finish the toolset for managing containers.

This automated configuration ensures the target instance is fully prepared with a custom-built container environment which is ready for the restore process when the checkpoint data is transferred to this instance at the end of migration.

IV.3.3. VPN and Load Balancer Setup. After the dependencies are installed on the target instance, the next step in the backend is setting up VPN tunnel between the source and the destination instance. Docker containers connect with each other using docker networking. It enables ability for containers to connect to and communicate with each other, or to non-Docker workloads. This creates a virtual network interface and assigns containers, their own private IP addresses. Migrating containers to the target system might break the communication in an inter-dependent container application. The containers should be able to communicate with each other even after the migration process on the target instance. To achieve this, we used a similar approach to [7]CloudHopper. This enables the containers to have the same IP address even after migration. Additionally the VPN tunnel between the source and the target instances brings the added benefit of secure transfer of checkpointed files. HAProxy (an industry standard load balancer) is also set up on both the source and target machines, in order to be able to route incoming traffic to the active container during migration after migration. Along with this, HAProxy

contains a queue, which keeps pending requests until the target service is available.

IV.3.4. Migration Process. The migration process is then executed using newly implemented Docker commands that accept specific flags, such as `predump` (a boolean value) and `parent_dir` (the path to the previously checkpointed image). The command is executed iteratively, continuously checkpointing until the delta value—representing the difference in size between successive checkpointed images reach a dynamically set threshold. Algorithm 1 provides the pseudo code for the method being used to calculate the dynamic threshold in order to determine the point of final checkpoint. Once this condition is met, the migration is finalized by stopping the instance at the source and starting it at the destination. In the next subsection we discuss this algorithm in more detail.

V. Algorithm for Optimal Cloud Service Provider Selection

V.1. Overview

The algorithm is designed to facilitate the selection of CSPs by analyzing parameters such as latency, bandwidth, and geographic location. It aims to streamline the target CSP selection process for the users by considering criteria and preferences set by them, resulting in a reduced search space. This approach ensures that enterprises can tailor their cloud strategies to meet specific operational goals, whether they prioritize cost efficiency, performance, or a balance of both.

V.2. Steps Involved

V.2.1. Step 1: Data Collection. The algorithm begins by collecting data on latency and pricing from various CSPs. It pings each server in the cloud to measure latency and uses CSP APIs to retrieve current pricing information. This data collection is crucial for understanding the current performance and cost landscape of each provider. By leveraging real-time data, the algorithm can provide up-to-date insights that reflect the latest market conditions, enabling more accurate and informed decision making.

For permanent storage and high-performance querying, we leverage Google BigQuery as our data warehouse solution. The collected pricing and performance data is periodically synchronized to BigQuery tables, enabling complex analytical queries across historical data. This architecture allows us to maintain a comprehensive pricing history across all providers and regions.

The data collection service also implements a sophisticated caching system to optimize data retrieval and minimize API calls. At its core, the cache implementation utilizes a 24-hour lifetime to balance data freshness with API efficiency. We chose a JSON-based storage format to ensure cross-platform compatibility, implementing atomic file operations to prevent partial data reads during updates. A

timestamp-based invalidation system ensures data reliability while minimizing unnecessary refreshes.

Provider-specific data sources are handled through distinct integrations. AWS pricing data is collected through direct integration with the AWS Pricing API via boto3 (the official AWS library in python), allowing real-time access to instance pricing. The Azure integration leverages their Retail Prices API with built-in filtering capabilities, enabling precise data retrieval. GCP data collection utilizes their static pricing list while accounting for regional variations.

Parameter	Description	Source
Latency	Time delay in data transfer	Server Pings
Pricing	Cost of cloud services	CSP APIs
Bandwidth	Data transfer capacity	Network Analysis

TABLE 1. PARAMETERS AND THE SOURCE OF DATA COLLECTION

V.2.2. Step 2: Analysis and Comparison. Once data is collected, the algorithm analyzes the latency and bandwidth ranges to compare different CSPs. It evaluates the performance of each provider based on the latency measurements and pricing data, allowing for a comprehensive comparison. GCPing application is utilized to perform latency analysis on GCP regions. The request is sent to an EC2 instance which in turn uses the GCPing service to measure the latency to a particular region. Whereas for AWS and Azure, an endpoint is provided by the CSP in order to check latency.

The framework employs a multi-stage filtering system to ensure optimal resource selection. This analysis helps identify the strengths and weaknesses of each CSP, providing a clear picture of which providers offer the best value for specific use cases. The algorithm’s ability to process large volumes of data quickly ensures that comparisons are both thorough and efficient. Cost-based filtering enforces price thresholds while considering instance type equivalency across providers. The system accounts for reserved instance opportunities and volume discounts, creating a comprehensive cost model. This model normalizes pricing across providers, enabling accurate comparisons of seemingly disparate offerings. We maintain a dynamic map between the different instance naming conventions across the service providers.

V.2.3. Step 3: Automated Target Provisioning. The framework incorporates automated Infrastructure as Code (IaC) generation capabilities, specifically focusing on Terraform configuration generation. This feature bridges the gap between cost optimization recommendations and actual resource provisioning, enabling seamless deployment of recommended resources.

The Terraform generation module implements a template-based approach with dynamic variable substitution. This allows for flexible resource configuration while maintaining infrastructure consistency. The system accounts for provider-specific requirements, such as AMI selection, security group configuration, and SSH access setup. Additionally, it includes provisioning scripts for post-deployment

configuration, enabling automated setup of monitoring agents and required software packages. After the resources have been provisioned, a script is run on the target cluster that installs the required dependencies for the migration process. This is described previously in Section 4.3.2

V.3. Integration

The algorithm outputs the results in JSON format that can be easily integrated with existing cloud management tools. Along with this, the service integrates well and provides comprehensive connectivity options through RESTful endpoints for all major operations. Additionally asynchronous processing improves performance under load. The system implements thorough error handling and request sanity checks to maintain reliability and availability.

VI. Extending Pre-Copy Checkpoint Support for Docker

With version 1.25, Docker has introduced experimental checkpoint features. This allows Docker to utilize the container runtimes like containerd, and runc to perform checkpoint and restore operations. In order to achieve our goal of live migration with Docker containers, we needed to modify the existing section of codebase that dealt with checkpointing. Our task was now narrowed down to adding support for pre-dump for the docker cli and docker daemon. Upon a good look at the docker source code, we came to the understanding that, changes are not limited to the cli and daemon services alone. The docker daemon forwards all checkpointing requests to containerd which in turn makes a call to runc for the final checkpointing operation. Below is an in-depth look at the different components and the scale of changes that were required in order to add this functionality to the existing codebase.

VI.1. Docker CLI Modifications

The Docker Command Line Interface, uses docker checkpoint command to checkpoint running containers. The most significant changes here were the addition of a new boolean flag to enable pre-dump functionality, and a new flag for allowing developers to specify a parent directory for checkpoint file storage. The pre-dump flag will enable the user to perform a memory pre-dump while keeping the container still running after the operation. The parent-dir flag specifies the location of the previous pre-dump files (this is relative to the path of the current checkpoint). These modifications extend the existing command-line argument parsing mechanisms, and introduce several different checks to ensure that an illegal command is not entered. CLI communicated with the docker daemon via REST APIs due to which we needed to change the requests being issued at that particular endpoint as well. The particular struct was modified to include the two fields. Figure 3 explains the behavior and expected use-case for the different combinations of these flags.

Algorithm 1 Live Migration Algorithm

```
1: function LIVE_MIGRATION( $max\_iterations = 100, base\_threshold = 0, max\_threshold = 1000$ )
2:    $threshold \leftarrow base\_threshold$ 
3:    $iteration \leftarrow 0$ 
4:    $min\_iterations \leftarrow 3$ 
5:    $window\_size \leftarrow 3$ 
6:    $recent\_deltas \leftarrow EmptyList()$ 
7:    $prev\_size \leftarrow PERFORM\_PRE\_DUMP$ 
8:   while  $iteration < max\_iterations$  do
9:      $curr\_size \leftarrow PERFORM\_PRE\_DUMP$ 
10:     $delta \leftarrow |prev\_size - curr\_size|$ 
11:    Append  $delta$  to  $recent\_deltas$ 
12:    if  $length(recent\_deltas) > window\_size$  then
13:      Remove first element from  $recent\_deltas$ 
14:    end if
15:     $avg\_delta \leftarrow \sum recent\_deltas / length(recent\_deltas)$ 
16:     $\alpha \leftarrow 0.7$ 
17:     $threshold \leftarrow \min(max\_threshold, (1 - \alpha) \times threshold + \alpha \times avg\_delta)$ 
18:    if  $iteration \geq min\_iterations$  and  $delta < threshold$  then
19:      return ( $True, iteration, threshold$ )
20:    end if
21:     $prev\_size \leftarrow curr\_size$ 
22:     $iteration \leftarrow iteration + 1$ 
23:  end while
24:  return ( $False, iteration, threshold$ )
25: end function
```

pre-dump Flag	Parent-dir Flag	Action
No	No	Normal checkpoint, container stops running
Yes	No	First time doing a checkpoint for live migration, container keeps running
Yes	Yes	There was a previous pre-dump done, and we are doing another one.
No	Yes	There was a previous pre-dump done, and this is the final one.

Figure 3. Docker Pre-Dump Additional Flags and their importance

VI.2. Dockerd (Docker Daemon) Modifications

The Docker daemon (dockerd/moby) required extensive modification to accommodate the new pre-dump features. Due to the vastness of docker daemon, first step we did for this part was to trace out each function and modules that were involved, once the checkpoint process started. This was done by utilizing Delve, the industry standard debugger for golang and by log statements. This involved recompiling gRPC protocol buffer files and updating the API interface to support advanced checkpoint options.

The CheckpointCreate function was comprehensively modified to handle the new parameters, with careful implementation of parameter validation and error handling mechanisms to ensure reliable and secure container migration.

VI.3. Containerd Modifications

Containerd plays a crucial role in the container migration process. The team replaced the original runc checkpoint command with a more sophisticated version that supports pre-dump functionality. This involved adding support for new flags (pre-dump and parent-dir) modifying the gRPC call to include pre-dump checkpoint creation parameters. With the newly added flags the runc command is then executed iteratively thus indicating the start of the live migration process.

VII. Iterative Migration Algorithm

Algorithm 1 provides the overview of the mechanism used to determine a stopping point for pre-dumps, which essentially marks the start of downtime. It starts after the first checkpoint is performed (since it can be significantly larger than subsequent dumps). In order to minimize downtime of the application, it is necessary that we transfer majority of the data using pre-dumps and minimal amount of data on the final checkpoint. Since the application is active on the source even after a pre-dump, this will ensure a minimal amount of downtime. For achieving this, we use a sliding average

technique to maintain a dynamic average of the delta values encountered so far with a default window size of 3. Lines 1-7 initialize the different variables that will be used for the algorithm: *threshold* maintains the current dynamic threshold value this will be used as the predicate for the final checkpoint, *iteration* stores the current number of times pre-dump has been performed, *min_iterations*, *window_size*, *recent_deltas* are all used for storing the sliding average value. The iterative checkpoint process starts with the while loop along with the *iterations < max_iterations* condition that makes sure the loop always terminates. Inside the while loop, Lines 9-17 perform a pre-dump, store its value, calculates new *delta*, appends it to the list *recent_delta* and then updates the new threshold value by using alpha smoothening with $\alpha = 0.7$ by default. This ensures that we are not completely dependent on the delta size of the last 3 pre-dumps and we take into account the previous sizes as well. Finally we perform a check with the predicate *iterations $\geq$ min_iterations* and *delta < threshold*. If both these conditions become true, it implies we can now perform the final checkpoint, therefore we return from the current function.

During the process of checkpointing, minimal downtime is experienced by the end user, due to the presence of HAProxy at both target and source, ensuring high availability and seamless transition. HAProxy implementation maintains a queue of incoming requests in case the target service is not available immediately. Therefore when the target container becomes active, HAProxy forwards all the requests in the queue to the target application similar to [7].

VIII. Results

This section is divided into two parts, the first part discusses the detailed price differences between different instance types for different CSPs and the second part presents results from simulations of multiple different scenarios on which the service was tested

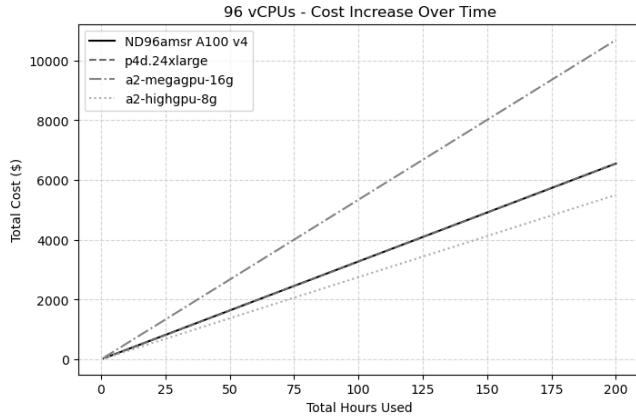


Figure 4. Docker Pre-Dump Additional Flags and their importance

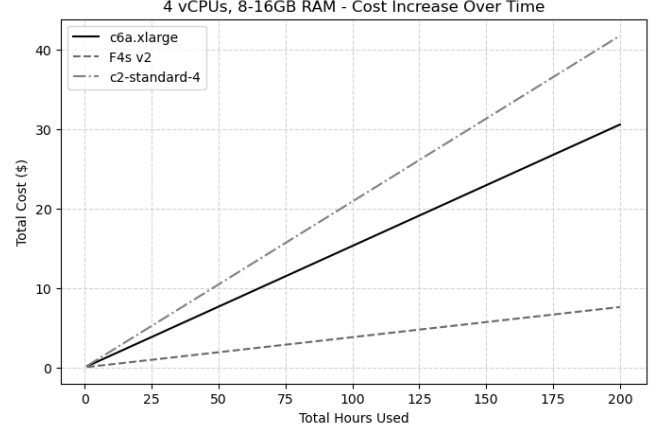


Figure 5. Docker Pre-Dump Additional Flags and their importance

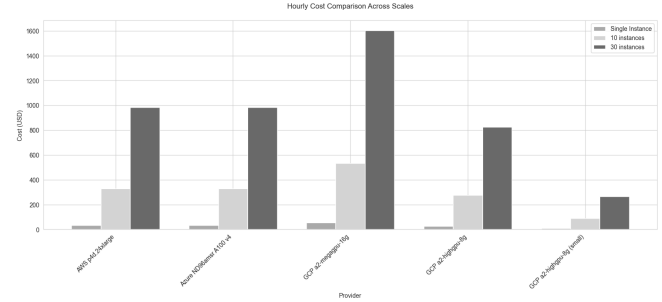


Figure 6. Cost for different number of instances

Figure 4 is a graphical plot of the hourly rates of accelerated compute instances for the top three Cloud Service Providers. The graph clearly indicates that the cost difference increases exponentially with the number of instances executed simultaneously, showing the potential savings of transferring to a lower-cost instance type. The figure also shows that GCP incurs the maximum cost of running instances, whereas AWS and Azure are charged the same at comparatively lower rates.

Figure 5 explores various compute instance choices that, although may seem cost-efficient in the short term, display dramatic cost disparities for extended time frames. Our findings show that the GCP a2-highgpu instance is the most costly of the cost-efficient choices, while the GCP a2-megagpu-16g displays a dramatic cost increase compared to its equivalents in Azure and AWS. These cost predictions, calculated by the algorithm automating the live migration process, illustrate the feasibility of the problem and justify the necessity of migration between Cloud Service Providers (CSPs).

The aforementioned cost predictions, calculated by the algorithm automating the live migration process, not only justify the problem's feasibility but also highlight the necessity of migration between Cloud Service Providers (CSPs). With cost-efficient migration strategies, organizations can optimize their cloud expenditures while ensuring perfor-

Scenario	Specifications	Iterations	1st Check-point (MB)	Avg. Check-point	Final Check-point
NodeJS Web Server	2 GB RAM Usage	5	540	175.00	127
Lightweight MongoDB	5 GB RAM, mix of reads/writes (10K TPS)	7	4,284	538.17	385
High-usage MongoDB	10 GB RAM, 40K TPS	7	8,446	1,308.33	1,004
Web App (Django)	2 GB RAM, low query volume	6	182	47.25	53
Kafka Queue	2 GB RAM, 10 topics, 500B msgs, 20K ops/sec	6	1,862	405.60	482
Redis Cache (3GB)	3 GB RAM, 2.3 GB data, 5K reads/sec	6	2,423	338.00	289
Redis Cache (5GB)	5 GB RAM, 4.6 GB data, 20K reads/sec	6	4,786	1,269.20	966
Jellyfin Media	4 GB RAM, 12 GB storage, 3 streams	8	2,420	3,043.85	2,915

TABLE 2. CHECKPOINT SIZES ACROSS DIFFERENT SCENARIOS

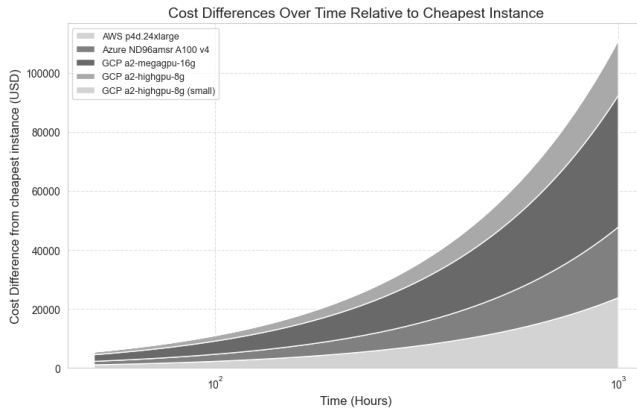


Figure 7. Cost Difference over time

mance and scalability.

IX. Conclusion

This paper introduces a novel approach for live migration of Docker containers across multi-cloud environments. The proposed solution is enhanced by an intelligent algorithm that dynamically selects the optimal cloud provider based on user-defined priorities, ensuring efficient resource utilization and minimal downtime. Furthermore, this research contributes to the Docker community by introducing open-source enhancements, particularly in command-line checkpointing support, facilitating more seamless container migration and improving overall system resilience.

Acknowledgement

The authors would like to acknowledge the monumental support from PES University, particularly the Department of Computer Science and Engineering, for providing the

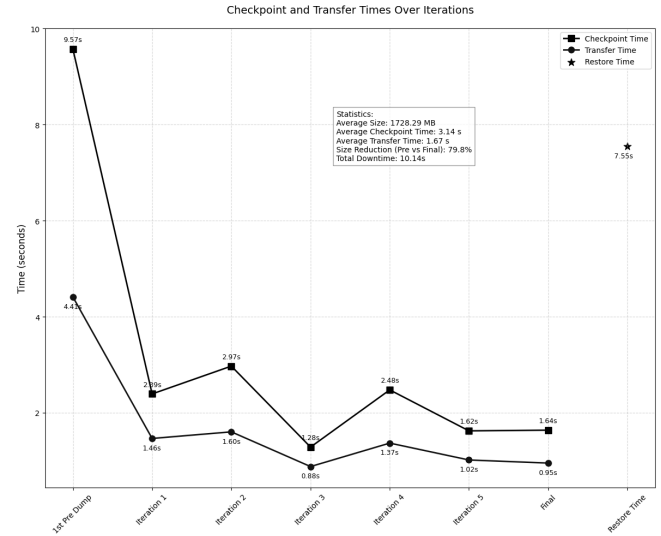


Figure 8. Cost Difference over time

necessary resources and guidance that helped in constructing this research paper.

The authors also extend their heartfelt gratitude to Dr. Prafullata K. Auradkar for her invaluable mentorship, encouragement, and insightful feedback throughout the course of this research.

References

- [1] S. Nadgowda, S. Suneja, N. Bila, and C. Isci, "Voyager: Complete Container State Migration," in *2017 IEEE 37th International Conference on Distributed Computing Systems*, IBM T.J. Watson Research Center.
- [2] Y. Takagawa and K. Matsubara, "Yet Another Container Migration on FreeBSD," Future University Hakodate.

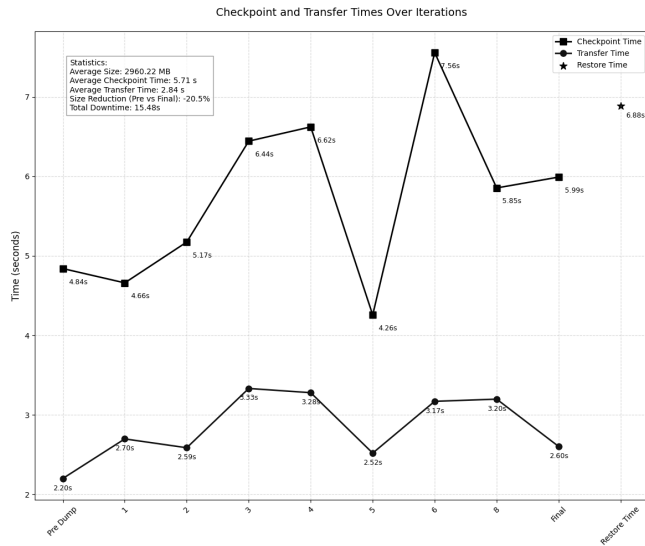


Figure 9. Cost Difference over time

- [3] J. Alonso, L. Orue-Echevarria, V. Casola, A. I. Torre, M. Huarte, E. Osaba, and J. L. Lobo, "Understanding the challenges and novel architectural models of multi-cloud native applications – a systematic literature review."
- [4] T. Fraundorfer, BSc, *Cloud Interchangeability*. Submission, Institute of Networks and Security. Thesis Supervisor: Assoc. Prof. Mag. Dipl.-Ing. Dr. M. Sonntag, Mar. 2021.
- [5] I. Odun-Ayo, Member, IAENG, F. Agono, and S. Misra, *Cloud Migration: Issues and Developments*, 2018.
- [6] R. Stoyanov and M. J. Kollingbaum, *Efficient Live Migration of Linux Containers*. University of Aberdeen, Aberdeen, UK.
- [7] T. Benjaponpitak, M. Karakate, and K. Sripanidkulchai, *Enabling Live Migration of Containerized Applications Across Clouds*. Department of Computer Engineering, Chulalongkorn University, Bangkok, Thailand.
- [8] B. Xu, S. Wu, J. Xiao, H. Jin, Y. Zhang, G. Shi, T. Lin, J. Rao, L. Yi, and J. Jiang, *Sledge: Towards Efficient Live Migration of Docker Containers*, in 2020 IEEE 13th International Conference on Cloud Computing (CLOUD). IEEE, 2020, pp. 321–328.